

JobHopper — Angel's presentation script

One speaker: **Angel**. Follow this top to bottom for the recorded video. Demo click paths match DEMO_RUNBOOK.md.

Hand-written — do not regenerate. SPEAKER_SCRIPT.md next to this file is rebuilt from the deck's speaker notes by scripts/make_script.py and will overwrite anything typed into it. This file is deliberately separate so a rebuild cannot destroy it. If the deck's narration changes, port the change across by hand.

Estimated total: 14:31 (target 14:00, hard ceiling 15:00 — the assignment caps the video at 15 minutes; the rubric penalises going over 20.)

Speaking estimate assumes 140 words per minute. The three demo slides are timed from the runbook rather than from word count.

Running order

#	Slide	Length	Ends at
1	Title	0:16	0:16
2	The goal	0:55	1:12
3	Functional requirements 1 of 2	0:45	1:57
4	Functional requirements 2 of 2	0:52	2:50
5	Non-functional requirements	0:48	3:38
6	Technology stack diagram	0:55	4:33
7	Why this stack	1:00	5:34
8	ERD — subject areas	0:40	6:14
9	ERD — full schema	1:21	7:36
10	User roles	0:36	8:12
11	Demo 1 — the visitor	1:30	9:42
12	Demo 2 — the registered user	2:15	11:57
13	Demo 3 — it really is a database	1:10	13:07
14	Summary — achieved vs changed	0:51	13:59
15	Close	0:32	14:31

Before you hit record

Do all of this once. Then leave the app and DB Browser ready and start on slide 1 of the deck.

- Start the backend **without** `--reload`, confirm `http://localhost:8000/health`.
- Serve the frontend on port **5500** (`http://localhost:5500/html/main.html`).
- Open DB Browser on `backend/jobhopper.db`, Browse Data tab, on a second desktop.
- Sign out / use a private window.
- Close editors, terminals, and file explorers (or park them on another desktop).
- Silence notifications. Screen at 1080p. Camera on if you can.
- Have a backup account ready in case live registration fails.
- Click through the full demo path once cold before the real take.

Full checklist lives in `DEMO_RUNBOOK.md`.

Delivery notes (Angel)

- **Look up.** Five rubric points ride on looking at the audience rather than reading. Learn the first and last sentence of each slide by heart; improvise the middle.
- **No code on screen.** Browser, DB Browser, and diagrams are fine. An editor, a terminal, or a source file is not.
- **Say the numbers.** 14 tables, 1,251 questions, 40 postings, 250 tests.
- **Do not narrate the mouse.** Say what a thing means, not where you are clicking.
- **If something breaks on camera,** say what should have happened and move on. Do not debug live; stop the recording and re-take instead.
- **Demo notation below:** lines in quotes are what you say. Numbered steps starting with **Do:** are what you do. Keep talking while you type — never narrate keystrokes.

Slide 1 — Title

0:00 - 0:16 • Est. length: 0:16

Hi, I'm Angel, presenting for the Laughing Bots — Rose, Elijah, Angel and Terrell. This is JobHopper: it reads real job postings, shows you which skills those postings are actually asking for, and lets you quiz yourself on any of them.

Slide 2 — The goal

0:16 - 1:12 • Est. length: 0:55

The problem isn't that this information is secret — job postings are public. It's volume. Search one title and you get forty postings, each with its own wall of requirements. The signal is which tools keep repeating, and nobody reads forty postings to find it. So people guess at what to learn next.

So our goal was this: analyse the job postings for a target role, identify the most in-demand skills, and show them as a word cloud — then help people actually build those skills through interactive Q&A games.

That second half is what we care about. Every skill in the picture is clickable. Click Python, and you're in a timed quiz on Python. It doesn't just tell you what to learn — it gives you somewhere to start.

Slide 3 — Functional requirements 1 of 2

1:12 - 1:57 • Est. length: 0:45

These are our functional requirements, written the way we specified them. I won't read them all — two things to point at.

In registration and login, it's the last pair: showing you your most recent game score, and your three most recent searches. Those are the requirements that forced everything to be stored against a real account rather than sitting in the browser.

And in word cloud creation, it's the last line. The size of each word is based on how often that skill appears across the postings we pulled. Size equals frequency — a count, not an opinion. That one sentence is the whole product.

Slide 4 — Functional requirements 2 of 2

1:57 - 2:50 • Est. length: 0:52

Job scraping is two requirements. Count how often each keyword appears — and display an error when there isn't enough job information to build a cloud. A thin, misleading picture is worse than saying we can't build one, so we made that a requirement rather than an edge case.

The Q&A game block is the exact path you'll see in the demo: click any word, get sent to that word's game, get questions matching the keyword and difficulty you chose, on a timer, with your score at the end.

One thing that isn't on the slide — every answer is locked in on the server before you're told whether it was right, so the running score and the final score can never disagree.

Slide 5 — Non-functional requirements

2:50 - 3:38 • Est. length: 0:48

Non-functional requirements — not features, but the app is wrong without them.

The top row is what we committed to in our spec: the cloud back within ten seconds, the game open within ten and scored within seven, and all user input validated and sanitised against injection. We get that last one structurally — every query goes through the ORM, parameterised, never built by string.

The bottom row we held ourselves to as we built. Passwords bcrypt-hashed and never returned, and a failed login gives the same generic message whether the account exists or not. The server decides every score, never the browser. And 250 tests, with nothing merging until both suites pass.

Slide 6 — Technology stack diagram

3:38 - 4:33 • Est. length: 0:55

The whole system on one page.

Top: the browser. Twelve HTML pages, Bootstrap, Sass, plain JavaScript modules — no framework, no build step. It talks to exactly one thing, our API, and anything tied to an account carries a signed token.

The middle band is where every rule lives: FastAPI, seventeen endpoints, Pydantic validating every request and response, and underneath it the services — hashing and tokens, skill extraction, and the ingest that keeps our data current. Note on the right that our postings are real, pulled from a live job-search API.

The bottom is one SQLite file, reached only through SQLAlchemy.

That line along the bottom is the rule that shaped everything: the browser never touches the database, and never decides anything it could be lied to about.

Slide 7 — Why this stack

4:33 - 5:34 • Est. length: 1:00

Why these choices.

Python and FastAPI: Python is the strongest shared language on this team, and FastAPI gave us validation for free plus live API docs — so the front end could build against a written contract instead of waiting for the backend.

SQLite: one file, no server to install, so four laptops and the CI runner run an identical database with zero setup — which removed a whole category of "works on my machine". SQLAlchemy keeps every query parameterised, so we're injection-safe by construction.

On the front end, Figma first. We designed every screen before anyone built it, which kept four people from producing four different-looking pages. Bootstrap gave us a responsive grid on day one, and Sass gave us one shared palette across twelve pages.

And GitHub Actions — the cheapest way to stop four people breaking each other's work.

Slide 8 — ERD — subject areas

5:34 - 6:14 • Est. length: 0:40

Before the full diagram, the shape of it. Fourteen tables in three groups.

Left: the job-market data — roles, postings, skills, and the junctions between them. That's where a cloud comes from. Middle: identity and activity. Right: the quiz engine.

What makes this one system rather than three is two shared tables. Skills is shared left to right — the same row that sizes a word in the cloud owns that word's question bank, which is why clicking a word can start a quiz. And users ties everything on the right to one account.

Slide 9 — ERD — full schema

6:14 - 7:36 • Est. length: 1:21

Trace each table with the cursor as you name it.

Let me trace the one path that touches most of it.

Start at ROLES. One role has many JOB_POSTINGS — title, company, location, salary range, date posted.

Now the interesting part. A posting mentions many skills, and a skill appears in many postings. That's many-to-many, which a relational database can't store directly — so JOB_SKILLS resolves it. Its primary key is the pair, job plus skill, which is what guarantees one posting can only count once toward a skill.

So the word cloud is one query: the postings for this role inside the date window, joined through job_skills, counted per skill, sorted descending. That count is the word size.

Follow SKILLS right and it becomes the quiz. One skill has many QUESTIONS split by difficulty, each with its ANSWER_OPTIONS — one flagged correct, and that flag never leaves the server.

Starting a quiz creates a QUIZ_SESSION recording which questions went out and what you picked — that is what makes live scoring trustworthy, and what stops a finished quiz being replayed. Submitting lands the result in GAME_ATTEMPTS.

And down the middle, USERS — every search and every attempt hangs off it.

Slide 10 — User roles

7:36 - 8:12 • Est. length: 0:36

This slide sets up the live demo. After you finish speaking, share your whole screen and switch to the browser with the home page already open.

One note before the demo, because it shapes what you'll see.

JobHopper has two roles, and the difference is whether the request carries a valid token. A visitor can read the public pages and register — nothing else. A signed-in user gets word clouds, quizzes that save their result, and a profile.

And there's deliberately no admin role. Postings come in through the ingest pipeline and questions load from a fixture, so there was never a job an admin would log in to do.

Slide 11 — Demo 1 — the visitor

8:12 - 9:42 • Est. length: 1:30

Share your whole screen now. Browser only — no terminal, no editor.

STEP-BY-STEP

1. **Do:** Home page already open (`main.html`), signed out.

"This is JobHopper the way anyone arrives at it — not signed in. Rules, stack and creators are open to everybody."

2. **Do:** Click Game Rules. Don't scroll far.

"Ten questions, three difficulties, each with its own clock."

3. **Do:** Try to open the word cloud page directly while still signed out.

"Now watch what happens if I go straight for a word cloud with no account. It sends me to sign-in — and that's not the page hiding a button. The API refuses the request too, so there's no way around it."

4. **Do:** Click Sign Up. Type a 3-character username so the validation fires.

"So let's make one. Validation is inline, right where the problem is."

5. **Do:** Fix the username, then fill the rest. Job title and location are type-ahead — start typing and real options drop down. Keep talking while you type.

"The same form takes your first search — and those suggestions are the job titles and locations we actually have postings for."

6. **Do:** Submit. You should land signed in on your first word cloud.

Fallback if registration fails: sign in with the backup account, say "we've already got one set up", and carry on. Do not debug on camera.

Slide 12 — Demo 2 — the registered user

9:42 - 11:57 • Est. length: 2:15

This is the main event. Stay in the browser. You must finish all ten questions — the score only saves when the quiz submits (tenth answer or timer out). Leaving early empties game history on the profile.

STEP-BY-STEP

1. **Do:** You are on the cloud registration just built. Point at the biggest words.

"Here's what that search produced. The biggest words are the ones that appeared in the most postings for that role — that's a count, not our opinion. Anything we have questions for is clickable."

2. **Do:** Click one of the biggest skills.

"So let's take that one."

3. **Do:** On the difficulty screen, choose **EASY** (three minutes — enough to finish all ten).

"Three difficulties, three timers. I'll take easy — three minutes."

4. **Do:** Answer question 1 correctly.

"It tells me straight away whether I got it. It can do that safely because my answer is already locked in on the server before I'm told — so knowing it now can't change what I picked."

5. **Do:** Answer question 2 wrong, on purpose.

"And there's the other case. It shows me the right answer, and the clock pauses while I'm reading it."

6. **Do:** Click through questions 3 to 10 without narrating each one (~60 seconds if you don't read them aloud).

"I'll speed through the rest of these."

7. **Do:** After the tenth answer, the Submit button becomes a return button. Click it — you land on the profile.

"That's graded and saved. This is my profile — the search I ran is under recent word clouds, and the quiz I just played is under game history, score and all. We didn't refresh anything, and none of this is hard-coded. That's the database answering."

8. **Do:** Click Search Again on the saved cloud.

"And any saved search re-runs in one click."

9. **Do:** Click **My Skill Roadmap** on the profile.

"Same data, asked a different way. These are the eight skills most in demand for that role, in order, and I can mark where I am on each one."

10. **Do:** Set one step to **In progress**, then scroll down to **Jobs Hiring For This Role**.

"And underneath, the postings that ranking came from — company, location, a link out to the real listing. The order isn't our opinion, it's what these jobs are asking for."

Running long? Steps 9 and 10 are the safe cut — about twenty seconds. The roadmap still appears on the summary slide either way. Everything up to step 8 is required.

Slide 13 — Demo 3 — it really is a database

11:57 - 13:07 • Est. length: 1:10

Switch to the DB Browser window already open on the second desktop. Do not open a terminal — that reads as code. This part is worth 9 rubric points — do not rush it.

STEP-BY-STEP

1. **Do:** Show the table list in the sidebar.

"Last thing — proof this is really sitting in a database. There are our fourteen tables, the same fourteen from the ERD. That diagram isn't what we planned; it's what's in the file."

Note: DB Browser lists fifteen. The extra one is sqlite_stat1 (SQLite's own stats). Always say "our fourteen tables".

2. **Do:** Browse Data → **searches**. Scroll to the last row.

"The bottom row is the search from ninety seconds ago — role, salary, shape, timestamp, and a user_id pointing at the account we made on camera."

3. **Do:** Switch to **game_attempts**.

"Same for the quiz. Skill, difficulty, score, seconds taken."

4. **Do:** Switch to **users**. Point at the password_hash column.

"And this is the users table. That's a bcrypt hash. We couldn't tell you that password if we wanted to — we never stored it."

5. **Do:** Switch to **job_postings**.

"These are the forty listings everything is built on — the same ones you saw on the roadmap page. Every word in that cloud is a count across these rows."

6. **Do:** Switch to **skills** or **job_skills**.

"And this is where the cloud actually comes from — a count per skill, not a list we typed out."

7. **Do:** Stop sharing the DB Browser / return to the deck.

"Everything you just saw came out of that one file."

Slide 14 — Summary — achieved vs changed

13:07 - 13:59 • Est. length: 0:51

The honest accounting.

On the left, what worked. Everything you just saw, plus a question bank that beat its own target — we aimed for ten per skill per difficulty and shipped fifteen, so more than 1,250 questions. And 250 tests on every pull request.

On the right, what changed. The top two are the same story: the external API limited what we could reliably fetch, so typing any role became four supported roles, and location became a menu of places that actually have postings behind them. Both were us refusing to offer a search that comes back empty.

The rest is choosing depth over breadth — and one place where our documentation got ahead of the app, which is on us.

Slide 15 — Close

13:59 - 14:31 • Est. length: 0:32

Three things next. Rank the roadmap by what you're weak at, not just by what the market wants. Widen the ingest; the pipeline isn't the limit, the skill vocabulary is. And the one we actually want: close the loop, so the skills you keep getting wrong become the ones your cloud puts in front of you.

Right now the two halves of the app share a database. They should share a memory.

That's JobHopper. Thank you.